

Caffe2-Plan Rust Crate

The `caffe2-plan` crate is part of the workspace containing the Rust translation of the Caffe2 operator library. This crate provides Rust implementations of classes and functions related to execution plans for Caffe2 networks. These execution plans are used to perform efficient and optimized execution of neural networks on various hardware platforms.

The `caffe2-plan` crate includes several important classes such as `ExecutionStepWrapper`, `CompiledExecutionStep`, `CompiledGuard`, and `WorkspaceIdInjector`. The `ExecutionStepWrapper` class wraps a Caffe2 network `ExecutionStep` object and provides a Rust interface for executing the step on a given workspace. The `CompiledExecutionStep` class provides an interface for executing a compiled version of a `ExecutionStep`. The `CompiledGuard` is a utility class for guarding the compiled step, and `WorkspaceIdInjector` injects a workspace ID into a network's execution plan.

The crate also includes several utility functions such as `cancel`, `execute_step_recursive`, `get_continuation_test`, `get_should_stop`, `reset`, `reset_with_compiled_ref`, and `rethrow_exception`. These functions provide additional functionality for executing and managing Caffe2 network execution plans.

The `caffe2-plan` crate is in the process of being translated from C++ to Rust, so it is possible that some of the function bodies are still in the process of translation.

Overall, the `caffe2-plan` crate is a crucial component of the Caffe2 operator library in Rust, providing efficient and optimized execution of neural networks on various hardware platforms.

32> what is an execution plan? how do we specify one of these more generally?

An execution plan is a specification of the sequence of operations that need to be executed in order to achieve a specific goal. In the context of the `caffe2-plan` crate, the execution plan specifies the sequence of operators that need to be executed in order to produce the desired output.

Execution plans can be specified in various ways depending on the problem domain. For example, in machine learning, an execution plan can be specified as a directed acyclic graph (DAG) where nodes represent operations (such as matrix multiplication, convolution, or activation functions), and edges represent the flow of data between operations. The execution plan is then the sequence of operations that need to be executed in a topological order of the DAG.

In the `caffe2-plan` crate, execution plans are specified using a network definition (NetDef) format, which is a protocol buffer definition that describes the computational graph. NetDefs are parsed and translated into a data structure that represents the execution plan, and this plan can then be executed to produce the desired output.

33> so, the execution plan is the same thing as the dag?

Yes, that's correct. The execution plan is essentially the directed acyclic graph (DAG) of operators that need to be executed in order to carry out a computation. It specifies the order in which the operators should be executed, and any dependencies that they have on each other.

34> is it the execution plan for a single thread? or several?

The execution plan in Caffe2 can be designed for both single-threaded and multi-threaded execution. It depends on the configuration and resources available. The execution plan can be designed to be parallelized across multiple threads, using thread pools or other techniques, to take advantage of multiple CPU cores or other hardware resources.

35> does the execution plan specify what one single thread does? or is it something which needs to be scheduled to different threads?

The execution plan specifies the sequence of operations that need to be performed in order to complete a task. However, the actual execution of these operations can be parallelized across multiple threads. In other words, the execution plan can be used to divide the workload among different threads, but it does not dictate how the threads should be scheduled or how the tasks should be distributed. The scheduling and distribution of tasks among threads is typically handled by a separate runtime system or framework.

36> what else should we know about this crate?

The `caffe2-plan` crate contains functionality related to executing a computation plan specified using the Caffe2 operator library in Rust.

The `CompiledExecutionStep` struct represents a compiled execution step that is ready for execution. The `ExecutionStepWrapper` wraps an execution step and provides a layer of indirection that allows the user to replace an execution step with a new one at runtime.

The `WorkspaceIdInjector` struct is responsible for injecting a workspace ID into a plan during its execution. This is useful when running multiple plans on the same workspace concurrently.

The `ReporterInstance` struct represents an instance of a `Reporter` and is used to report progress during plan execution. The `Reporter` trait defines a set of methods that can be used to report the status of a plan during execution.

The `cancel` function can be used to cancel the execution of a plan.

The crate also includes several error-related structs and functions, including `ExceptionWrapper`, `ExceptionWrapperTerminate`, `TestError`, `BlockingErrorOp`, `ErrorOp`, `StuckAsyncOp`, and `StuckBlockingOp`. These are used to handle errors that occur during plan execution.

Finally, the crate includes several test-related functions, such as `plan_executor_test_blocking_error_plan`, `plan_executor_test_empty_plan`, `plan_executor_test_error_async_plan`, and `plan_executor_test_should_stop_with_cancel`. These functions can be used to test the functionality of the crate.